



ALARM CLOCK

Python Mini Project



Submitted By:	DEV
Registration Number:	26BCY10129
Branch:	B.Tech CSE [CYBER SECURITY AND FORENSIC EXPERT]
Course:	Introduction to Problem Solving in Python
Institution:	VIT Bhopal University
Academic Year:	2026 - 2027

1. Introduction

The Alarm Clock is a simple desktop application developed in Python using the Tkinter library. It allows the user to select an hour, minute and second, then starts a background thread that checks the current system time. When the selected time is reached, a WAV sound file is played.

Objectives

- Create a graphical user interface for setting an alarm.
- Use Python date and time functions to compare the current time with the selected time.
- Play an alarm sound automatically at the correct time.
- Use threading so the graphical interface remains responsive while the alarm is running.

Tools and Technologies

Technology	Purpose
Python	Main programming language
Tkinter	Graphical User Interface (GUI)
datetime	Reads the current system time
time	Adds a one-second delay between checks
winsound	Plays the WAV alarm sound on Windows
threading	Runs the alarm loop without freezing the GUI
os	Builds the path to the sound file

2. Working of the Project

Step 1: The program creates a Tkinter window titled as an Alarm Clock interface.

Step 2: Three drop-down menus allow the user to choose hour, minute and second.

Step 3: Clicking **Set Alarm** starts a separate thread.

Step 4: The alarm function reads the selected time and compares it with the computer's current time every second.

Step 5: When both times match, the program prints **Time to Wake up** and plays the selected WAV file.

Program Flow

START
Create Tkinter window
Select Hour : Minute : Second
Click Set Alarm
Start alarm thread
Read current system time
Current time = Alarm time?
NO → Wait 1 second and check again
YES → Play WAV alarm sound
END / Close application

3. Python Source Code

```
from tkinter import *
import datetime
import time
import winsound
from threading import *
import os

root = Tk()
root.geometry("400x200")

SOUND_FILE = os.path.join(
    os.path.expanduser("~"),
    "Downloads",
    "salamisound-5673860-basic-call-vibration (1).wav"
)

def Threading():
    t1 = Thread(target=alarm)
    t1.start()

def alarm():
    while True:
        set_alarm_time = f"{hour.get()}:{minute.get()}:{second.get()}"
        time.sleep(1)
        current_time = datetime.datetime.now().strftime("%H:%M:%S")
        print(current_time, set_alarm_time)

        if current_time == set_alarm_time:
            print("Time to Wake up")
            winsound.PlaySound(
                SOUND_FILE,
                winsound.SND_FILENAME | winsound.SND_ASYNC
            )

Label(root, text="Alarm Clock",
      font=("Helvetica 20 bold"), fg="red").pack(pady=10)

Label(root, text="Set Time",
      font=("Helvetica 15 bold")).pack()

frame = Frame(root)
frame.pack()

hour = StringVar(root)
hours = tuple(f"{i:02d}" for i in range(24))
hour.set(hours[0])
OptionMenu(frame, hour, *hours).pack(side=LEFT)

minute = StringVar(root)
minutes = tuple(f"{i:02d}" for i in range(60))
minute.set(minutes[0])
OptionMenu(frame, minute, *minutes).pack(side=LEFT)

second = StringVar(root)
seconds = tuple(f"{i:02d}" for i in range(60))
second.set(seconds[0])
OptionMenu(frame, second, *seconds).pack(side=LEFT)

Button(root, text="Set Alarm",
      font=("Helvetica 15"),
      command=Threading).pack(pady=20)

root.mainloop()
```

4. Code Explanation

Imports: Tkinter creates the interface; datetime gets the current time; time provides the delay; winsound plays the alarm; threading prevents the GUI from freezing; os creates the sound-file path.

SOUND_FILE: The sound file is stored in the Windows Downloads folder. The file must exist at this location and must be a compatible WAV file.

Threading(): Creates a new thread and runs the alarm() function in the background.

alarm(): Continuously builds the selected alarm time, reads the current time in HH:MM:SS format, and compares both values. When they match, winsound.PlaySound() starts the alarm audio.

OptionMenu: The three drop-down menus accept valid 24-hour clock values: 00-23 for hours and 00-59 for minutes and seconds.

Set Alarm button: Calls Threading(), which starts monitoring the selected time.

Expected Output

A 400 × 200 desktop window appears with the heading **Alarm Clock**, a **Set Time** label, three drop-down menus, and a **Set Alarm** button. At the selected time, the program plays the WAV sound.

5. Conclusion

This project demonstrates the practical use of Python GUI programming, date and time handling, multithreading, file paths and audio playback. The project is small and easy to understand, while showing how multiple Python modules can work together to build a useful desktop application.

Future Improvements

- Add a Stop Alarm button.
- Allow multiple alarms.
- Add snooze functionality.
- Display the live current time in the window.
- Let the user choose the alarm sound from the application.
- Add a cleaner modern interface and save alarms between sessions.

Project Title: Alarm Clock

Submitted By: DEV | **Registration No.:** 26BCY10129

VIT Bhopal University | Academic Year 2026 - 2027